

TÀI LIỆU THIẾT KẾ DATABASE

Kiến trúc CSDL Hệ Thống Đặt Món Ăn Trực Tuyến - Chuẩn Production

1. Nền tảng cấu trúc (Base Architecture)

Hệ thống sử dụng cơ chế kế thừa `BaseModel` để tái sử dụng mã nguồn và chuẩn hóa cấu trúc dữ liệu cho toàn bộ các bảng trong cơ sở dữ liệu.

- Tính nhất quán:** Mọi model đều kế thừa các trường `id`, `created_at`, và `updated_at`. Điều này giúp dễ dàng truy xuất thời gian tạo và cập nhật của bất kỳ bản ghi nào mà không cần khai báo lại.
- Cơ chế Soft Delete:** Thuộc tính `is_active = True/False` được sử dụng để thực hiện xóa mềm. Thay vì sử dụng lệnh `DELETE` trong SQL gây mất mát dữ liệu vĩnh viễn, hệ thống chỉ ẩn dữ liệu đi. Điều này đặc biệt quan trọng để giữ lại lịch sử giao dịch cho các mô hình Machine Learning phân tích sau này.

2. Ánh xạ Yêu cầu Nghiệp vụ (Business Logic)

2.1. Quản lý Người dùng và Phân quyền

Bảng `User` lưu trữ thông tin khách hàng, chủ nhà hàng và quản trị viên, được phân tách rõ ràng qua `RoleEnum`. Thiết kế này giúp kiểm soát truy cập (Authorization) dễ dàng ở tầng Middleware.

2.2. Danh mục và Nhà hàng

Để đáp ứng yêu cầu "Tìm kiếm và duyệt nhà hàng", các trường quan trọng như `name` của `Restaurant` và `Dish` đều được thiết lập `index=True`. Việc đánh chỉ mục này giúp tăng tốc đáng kể các truy vấn tìm kiếm bằng từ khóa (Full-text hoặc LIKE queries).

2.3. Quản lý Giỏ hàng

Giỏ hàng được chia thành `Cart` (đại diện cho một giỏ duy nhất của một User) và `CartItem` (các món ăn bên trong). Thuộc tính `cascade="all, delete-orphan"` đảm bảo tính toàn vẹn dữ liệu: khi một giỏ hàng bị hủy, toàn bộ các món ăn tạm thời bên trong nó cũng được tự động dọn dẹp khỏi cơ sở dữ liệu.

3. Xử lý Đơn hàng & Thanh toán

Bảo toàn tính lịch sử về giá: Yếu tố cốt lõi của một hệ thống thương mại điện tử là bảng `OrderItem` tách riêng trường `price_at_purchase`. Giá món ăn có thể thay đổi theo thời gian, nhưng việc lưu lại giá ngay tại thời điểm chốt đơn đảm bảo tổng tiền của hóa đơn trong quá khứ không bao giờ bị sai lệch.

Quản lý vòng đời đơn hàng và thanh toán được ràng buộc nghiêm ngặt bằng **Enums** (`OrderStatusEnum`, `PaymentStatusEnum`). Thiết kế này đóng vai trò như một bộ State Machine, ngăn chặn việc dữ liệu bị cập nhật các trạng thái không hợp lệ (ví dụ: lỗi đánh máy từ phía Frontend hoặc tấn công thay đổi tham số).

4. Thiết kế tối ưu cho Hệ thống Gợi ý & AI

Để đáp ứng yêu cầu gợi ý món ăn, dự đoán và phân tích đánh giá từ đề bài, CSDL đã được thiết kế sẵn các Feature Column phục vụ trực tiếp cho việc trích xuất và huấn luyện mô hình:

- **Gợi ý theo vị trí (Location-based):** `latitude` và `longitude` tại `User` và `Restaurant` cho phép áp dụng các thuật toán tính khoảng cách (như Haversine) để đề xuất nhà hàng gần nhất.
- **Collaborative / Content-Based Filtering:** Trường `taste_preferences` (User) và `flavor_tags` (Dish) cho phép lưu trữ dưới dạng mảng Text hoặc JSON. Dữ liệu này dùng để tính độ tương đồng (Cosine Similarity) giữa sở thích người dùng và đặc điểm món ăn.
- **Phân tích cảm xúc (Sentiment Analysis):** Bảng `Review` có sẵn trường `sentiment_score`. Các bình luận có thể được chạy qua mô hình NLP để chấm điểm thái độ (ví dụ 0.0 - 1.0). Khi hiển thị, hệ thống chỉ cần query các món có điểm số cao để làm nổi bật mà không cần phân tích lại văn bản.

5. Tối ưu hóa truy vấn (Performance Tuning)

Sử dụng chiến lược Lazy Loading linh hoạt của SQLAlchemy để giải quyết bài toán N+1 Query:

- **lazy='joined':** Dùng cho `CartItem` và `OrderItem`. Khi tải một đơn hàng, SQLAlchemy sẽ dùng `JOIN` SQL để gộp thông tin các món ăn trong 1 câu truy vấn duy nhất.
- **lazy='dynamic':** Dùng cho quan hệ `Restaurant -> Dishes`. Giúp trả về một Query Object thay vì tải toàn bộ hàng trăm món ăn vào bộ nhớ RAM. Ứng dụng có thể thoải mái kết hợp các bộ lọc hoặc phân trang `.paginate()` trước khi thực thi.